

# ATANDA Command Centre — PromptWare Design Document

---

Investor Edition · v1.0 · Codebase-of-record snapshot

*A PromptWare Design Document (PWDD) is the certified, post-ingestion outcome of a FORGE.BONSAI HARNESS session. This PWDD certifies the complete Atanda Command Centre codebase as it stands today: an authenticated portal in front of an 8-stage AI prompt engineering pipeline, billed on a tiered Stripe subscription, deployed on Replit.*

---

## 1. Executive Summary

---

**One sentence.** Atanda Command Centre turns vague product ideas into deploy-ready software by running them through an 8-stage, deterministic prompt-engineering pipeline that ends with a downloadable, certified codebase.

**Who it is for.** Founders, product managers, and engineering leaders who already use ChatGPT, Claude, or Cursor but can't get audit-grade output. The Command Centre replaces "vibe-coding into a wall" with a contract-first pipeline that produces traceable artefacts at every step.

**What we sell.** A SaaS subscription (4 tiers, \$0 !' enterprise) plus a per-project one-time ingestion credit. The subscription gates access to the higher engines (F5, F6, F7, F8); the credit pays for ingesting an existing design document into the pipeline.

**Why it works.** Every other LLM tool gives you one shot at one prompt. We give you an instrument panel: 8 ordered engines, each one auditable, each one constrained by a system prompt that has been hardened to v2 with anchored rubrics, calibrated escalation, and audit-grade math. The pipeline produces — SPARTAN-class A/B/C bundles with a public verification URL — that we can stand behind.

**Where we are.** Codebase complete end-to-end. F1 through F8 ship. Stripe billing is live in test mode. Demo mode is wired for investor previews. The deployment story is one click on Replit. The build is contract-first (OpenAPI single source of truth) and typechecks clean across every package in the monorepo.

**Why now.** The first wave of AI tooling has trained the market on the problem (hallucination, drift, untraceable output). Buyers are now ready to pay for the cure. The Command Centre is the cure shaped as a product.

---

## 2. The Problem

---

The current AI-assisted product workflow looks like this:



- A founder has an idea.
2. They paste a paragraph into ChatGPT or Claude.
  3. They get back something that *looks* like a spec.
  4. They paste *that* into Cursor or Replit Agent.
  5. They get back something that *looks* like working code.
  6. Three days later, nothing actually works, and they cannot explain why.

The root cause is not the models. It is the *missing instrumentation between the model and the user*. There is no contract, no rate-limit, and no escalation path. Every session is an unrepeatable performance.

The Command Centre fixes this by being the instrumentation.

---

### 3. The Solution — The FORGE.BONSAI HARNESS

---

The HARNESS is a sequence of **atomic prompts** (F1 through F7, plus two side engines — F6-VDJ and F8 Code DJ — plus a Digital Evolution path called DE-SPC). Each engine has:

- a hardened system prompt with explicit operating rules,
- a typed input contract validated by generated Zod schemas,
- a typed output contract validated by Zod and persisted as a database artefact,
- a tier gate (Explorer / Practitioner / Architect / Institution),
- a daily rate limit counter,
- per-call telemetry written to [harness\\_engine\\_runs](#) for cost analytics,
- a defined hand-off to the next engine in the pipeline.

The user runs *one coherent session* per product idea. Each engine consumes the certified output of the previous one. The final stages produce a SPARTAN-certified MVP PDD with a public verification URL and (for Architect tier) a complete deploy-ready codebase.

---

### 4. The 8-Stage Pipeline

---

#### F1 — Prompt Diagnostic

Takes a raw, messy founder prompt and diagnoses it across 7 craft pillars (Specificity, Reasoning, Iteration, Decomposition, Format, Examples, Context). Emits a structured **JCSE breakdown** (Joint Context Score Estimator) and an upgraded "Atomic Prompt" candidate.

**Tier:** All tiers. **Rate:** Explorer 5/day, Practitioner 50/day, Architect/Institution unlimited.

#### F2 — Atomic Prompt

Refines the F1 output into a true **Atomic Prompt** — a self-contained,

shared. Continues the JCSE rubric context-rich prompt that can be re-used and **Context Craft Mini-Quest Badges** for any wilds that exceeds the "precise" band (sub-score "e 6).

**Tier:** All tiers. **Rate:** Explorer 3/day !' Architect unlimited.

### F3 — MA Birth Package (streaming)

Grows the Atomic Prompt into a **Micro Agent Birth Package** — a fully populated agent specification (role, constraints, tools, examples, escalation rules). Runs as a server-sent event stream so the user can watch the agent take shape in real time.

**Tier:** All tiers. **Rate:** Explorer 1/day !' Architect unlimited.

### F4 — Micro PDD

Converts the MA Birth Package into a **Micro Product Design Document** — the first qualifying design artefact. This is the gate point between "having an idea" and "having a specification".

**Tier:** All tiers. **Rate:** Explorer 1/day !' Architect unlimited.

### F5 — SPC (System-Prompted Constellation)

Practitioner tier and above. Takes the Micro PDD and builds a **full SPC** — a complete, multi-prompt constellation that can drive an entire feature build. Interactive: the engine asks clarifying questions before finalising.

**Tier:** Practitioner+. **Rate:** Practitioner 5/day !' Architect unlimited **Escalation bypass:** if the user holds an active escalation on the session, the tier gate is waived for that session.

### F6 — ATLAS PDD

Practitioner tier and above. Drafts a **4-Part ATLAS PDD**: *cheatSheet · execSummary · worksheet · implementation*. This is the ATLAS-shaped Project Definition Document that production teams actually hand to engineering.

**Tier:** Practitioner+. **Rate:** Practitioner 5/day !' Architect unlimited.

### F6-VDJ — Vibe DJ (side capability)

Reads the ATLAS PDD and recommends an **IDE and coding "vibe"** best suited to actually building it (e.g. Cursor + TypeScript-strict + monorepo, or Replit Agent + Next.js + Vercel). Advisory only — does not produce a stored artefact and does not advance the pipeline.

**Tier:** Practitioner+.

### F7 — SPARTAN Compressor (streaming)

The certification engine. Takes the ATLAS PDD and runs it through the **7-step SPARTAN SCM**

(SCAN !' PROFILE !' ASSESS !' REDUCE !' TRANSFORM !'

ZPOS+5 !' PACKAGE). The output is a **SPARTAN-certified MVP PDD** classed A, B, or C, with a unique cert ID and a public [/verify](#) URL. Email is sent to the operator on issuance.

**Tier:** Practitioner+. **Rate:** Practitioner 3/day !' Architect unlimited.

## F8 — Code DJ \*(new — Architect tier only)\*

Reads a SPARTAN-certified MVP PDD / PWDD and a target platform selection ([nextjs-vercel](#), [react-vite-static](#), [express-replit](#), [expo-mobile](#), [pnpm-monorepo](#)) and emits a **complete codebase scaffold**: up to 12 files

plus a manifest with entrypoint, install command, run command, build

command, and deploy target. The scaffold is persisted as a **CODEBASE\_BUNDLE** artefact and can be exported as a single JSON bundle for unpacking into any working tree.

Code DJ refuses to scaffold from any source that lacks a SPARTAN cert. The output schema caps file count at 12 and individual content at 40 KB, rejects path traversal, and forbids absolute paths.

**Tier:** Architect only. **Rate:** Architect 2/day, Institution unlimited. Explorer and Practitioner explicitly blocked.

## DE-SPC — Digital Evolution (gated side path)

Practitioner tier and above, plus the **ASPE quest badge** ("e3 SPCs and "e4 Micro Agents). Lets an experienced operator evolve an existing SPC [digitally\\_evolved](#) variant. The badge requirement ensures only practitioners with demonstrated craft can branch the canonical pipeline.

---

## 5. Architecture at a Glance

---

[illegible]

## 5.1 Architectural decisions worth flagging to investors

- **Contract-first.** Every endpoint is declared in OpenAPI first. The server validates inputs against generated Zod schemas; the client uses generated TanStack Query hooks. One source of truth, zero drift.
- **App-layer authorisation, not RLS.** Every database query that touches user-owned data filters by `req.localUser.id`. We did not bet the company on Postgres Row-Level Security as a net.
- **External-first account deletion.** When a user deletes their account we cancel Stripe first, delete Clerk second, and only then cascade the local row. This ordering prevents orphaned subscriptions and zombie tokens — a real failure mode in most SaaS products.
- **Idempotent Stripe webhook.** Every event id is recorded on receipt; replays return immediately without re-executing handlers. Stripe's legitimate retries are handled gracefully.
- **Per-engine telemetry is best-effort.** Token and cost accounting is written to `harness_engine_runs` on every Claude call but never breaks a user request. Telemetry outages are invisible to the operator.
- **Sentry is wired with safe defaults.** API uses `@sentry/node`, web uses `@sentry/react`. Both gate on DSN env vars and no-op cleanly in dev.

## 6. Tech Stack

| Layer       | Choice                                     | Why                                                 |
|-------------|--------------------------------------------|-----------------------------------------------------|
| Runtime     | Node.js 24, TypeScript 5.9                 | Modern async, strict types                          |
| Repo        | pnpm workspaces                            | Fast install, deterministic, monorepo-native        |
| API         | Express 5                                  | Maturity, middleware ecosystem, raw-body support    |
| Auth        | Clerk (Replit-managed tenant)              | OAuth + email + magic link, zero secret-handling    |
| Billing     | Stripe                                     | Industry standard, webhook + customer portal        |
| Database    | PostgreSQL                                 | ACID, JSONB for artefact content                    |
| ORM         | Drizzle                                    | Type-safe, no migrations folder in flux             |
| Validation  | Zod v4                                     | Runtime types, integrates with OpenAPI codegen      |
| API codegen | Orval (from OpenAPI)                       | One contract, two consumers                         |
| LLM         | Claude Sonnet 4 via Replit AI Integrations | No customer API key handling, billed through Replit |
| Front-end   | React 19 + Vite + Wouter + TanStack Query  | Fast HMR, lightweight router, mature data layer     |
| UI          | Tailwind CSS + shadcn/ui                   | Designer-trustable, component reuse                 |

|               |                                                     |                                                 |
|---------------|-----------------------------------------------------|-------------------------------------------------|
| Observability | pino logs, Sentry (API + web), per-engine telemetry | Logs, errors, and AI cost all visible           |
| Email         | Resend (with console-log dev fallback)              | Transactional, modern API                       |
| Deployment    | Replit                                              | One-click deploy, TLS handled, custom domains   |
| PDFs          | pdfkit + marked                                     | Investor-grade documents straight from markdown |

## 7. Business Model & Pricing

### 7.1 Tiers

| Tier         | Price        | F1-F4     | F5  | F6  | F7  | F6-VDJ | F8 Code DJ | DE-SPC          |
|--------------|--------------|-----------|-----|-----|-----|--------|------------|-----------------|
| Explorer     | Free         | Limited   | —   | —   | —   | —      | —          | —               |
| Practitioner | Subscription | Higher    | 5/d | 5/d | 3/d | yes    | —          | with ASPE badge |
| Architect    | Subscription | Unlimited | "   | "   | "   | yes    | 2/day      | with ASPE badge |
| Institution  | Enterprise   | Unlimited | "   | "   | "   | yes    | Unlimited  | with ASPE badge |

Prices live in Stripe (monthly + yearly per tier). Switching prices requires only a `STRIPE_PRICE_*` env var change — unknown price ids fail loudly with `UnknownPriceError`. Logging so launching a new tier is a checklist item, not a code change.

### 7.2 Per-project Ingestion Credit

Ingesting an existing design document (the IPDD — "Ingestion Product Design Document") into the pipeline is **billed per project, not per tier**. Any signed-in user — Explorer included — can buy one ingestion credit via a `mode=payment`). On `checkout.session.completed` (a row is inserted into `ingestion_credits` and atomically claimed on the `/api/ingest` call. On any failure path, the credit is released back `available` so a fluke does not burn the purchase.

This decouples ingestion revenue from subscription revenue and lets us land non-subscribers with a low-friction first purchase.

## 7.3 Revenue model summary

- **Recurring:** four-tier subscription, two billing cadences.
- **One-time:** per-project ingestion credit.
- **Enterprise:** Institution tier (negotiated, includes private deployment).
- **Defensible margin:** AI inference is billed through Replit's Integrations proxy at platform rates, with per-engine telemetry letting us reconcile unit economics nightly.

## 8. Defensibility & Moats

1. **Pipeline-as-product, not feature.** Most competitors sell a chat box. We sell a *pipeline*. The pipeline is the moat. Reproducing it requires not just the prompts but the contract-first instrumentation, the certification math (SPARTAN), the rate-limit topology, and the escalation graph.
2. **Certification surface.** Every certified MVP PDD has a public verification URL. As certificate volume grows, the verification corpus itself becomes a network effect — operators link to their certs in pitch decks, GitHub READMEs, and investor updates.
3. **Quest badges and skill graph.** Operators earn persistent badges (ASPE for craft, AISE for verified deployment, and seven Context Craft pillar badges). The skill graph creates retention pressure and gates higher-order capabilities like DE-SPC.
4. **Exemplar Library.** Hand-authored canonical examples drive new-user activation; a "Fork to session" CTA on every exemplar converts inspection into a started session in one click.
5. **The Code DJ asymmetry.** F8 turns a certified design document into a complete codebase. The defensibility is not the LLM call — it is that we *refuse to do this* without a SPARTAN cert. Code DJ is the reward at the top of the pipeline. Other tools generate code from prose. We generate code from a *contract*.

## 9. Traction Surfaces

### 9.1 Demo Mode

A `VITE_DEMO_MODE=true` build flag wires the front end into a no-auth investor preview that surfaces every workspace with seeded data. Full docs/[DEMO\\_PUBLISHING.pdf](#). This is how prospective investors and design partners get inside the product without an account.

### 9.2 Exemplar Library

Hand-curated SPCs and PDDs visible on the Command Deck. Every exemplar links to a "Fork to session" CTA that bootstraps a new session pre-loaded with the exemplar's artefacts. First-time activation tactic.

### 9.3 Quest Badges

| Badge family | Storage | Trigger |
|--------------|---------|---------|
|--------------|---------|---------|

|                   |        |                                                                        |
|-------------------|--------|------------------------------------------------------------------------|
| ASPE              | Live   | "e3 SPCs and "e4 Micro Agents authored                                 |
| AISE              | Stored | A certified MVP PDD's verify URL is reachable (with strict SSRF guard) |
| AISA              | Live   | Composite achievement across SPCs/MAs/PDDs                             |
| Context Craft (7) | Stored | Per-pillar JCSE sub-score "e 6 in any F1/F2 artefact                   |

Live badges re-compute on every `/me/badges` call. Stored badges live in `command_centre_badges` and `context_craft_badges` with idempotent upserts `GREATEST(bestScore, new)`.

## 9.4 Public verification surface

`/verify?certId=...` is unauthenticated and shows the cert class, issue date, and certifying session. This is the surface operators link to in the wild.

# 10. Security Posture

- **No customer-supplied AI keys.** All LLM calls go through Replit AI Integrations — we never handle or store an Anthropic key.
- **SSRF guard on verify URL.** The AISE URL verifier resolves the hostname and refuses RFC1918, loopback, link-local, cloud metadata (`169.254.169.254`), CGNAT, and multicast addresses. Only `https:` is allowed.
- **Strict JIT user sync.** First-time auth requires a successful `clerkClient.users.getUser` before inserting the local row. Clerk 404 produces 401. Other Clerk errors produce 503. Existing users short-circuit, so a Clerk outage cannot lock out provisioned accounts.
- **External-first account deletion.** Documented above. The ordering is load-bearing: flipping it would orphan Stripe subs and let stale tokens JIT-recreate shell users.
- **Idempotent webhook.** A duplicate Stripe event id returns `{ok: true, replay: true}` without re-executing handlers. Handler exceptions roll back the idempotency row so Stripe can retry legitimately.
- **Output schema hardening.** F8 Code DJ output Zod schema rejects path traversal, absolute paths, file count > 12, and content > 40 KB per file. The source artefact is required to be SPARTAN-certified before the LLM is even called.

# 11. Ontology

This is the terminology investors will hear from operators and should recognise on slides.

| Acronym | Expansion                         | Where it lives                                                  |
|---------|-----------------------------------|-----------------------------------------------------------------|
| IPDD    | Ingestion Product Design Document | Input the user already has, fed into <code>`/api/ingest`</code> |

|         |                                           |                                                  |
|---------|-------------------------------------------|--------------------------------------------------|
| PWDD    | PromptWare Design Document                | Certified output of an *ingested* session        |
| MVP PDD | Minimum Viable Product Design Document    | Certified F7 output of a *manual* session        |
| SPC     | System-Prompted Constellation             | Multi-prompt agent system (F5 output)            |
| MA      | Micro Agent                               | Single-agent specification (F3 output)           |
| JCSE    | Joint Context Score Estimator             | 7-pillar scoring rubric used by F1 and F2        |
| ATLAS   | Four-part PDD structure                   | F6 output shape                                  |
| SPARTAN | Seven-step certification compressor       | F7 engine                                        |
| VDJ     | Vibe DJ                                   | F6-VDJ recommender                               |
| Code DJ | Codebase DJ                               | F8 codebase scaffolder                           |
| ASPE    | Architect of SPCs & Pipelined Engines     | Earned quest badge                               |
| AISE    | Architect of Internet-Surface Engineering | Earned quest badge tied to a verified public URL |

*Important: the bare term "PDD" was historically overloaded — it meant **\*both\*** the human-authored input and the various engine outputs. The ontology now disambiguates: **\*\*IPDD\*\*** is always the input; **\*\*PWDD\*\***, **\*\*Micro PDD\*\***, **\*\*ATLAS PDD\*\***, and **\*\*MVP PDD\*\*** are all qualified outputs.*

## 12. Repository Topology

|                                    |                                               |
|------------------------------------|-----------------------------------------------|
| artifacts-monorepo/                |                                               |
| % % % artifacts/                   |                                               |
| % % % % api-server/                | Express 5 API, all engines, Stripe webhook    |
| % % % % command-centre/            | React+Vite web app: landing, /pricing, portal |
| % % % % mockup-sandbox/            | Internal canvas + component preview server    |
| % % % lib/                         |                                               |
| % % % % api-spec/                  | openapi.yaml – single source of truth         |
| % % % % api-client-react/          | Generated TanStack Query hooks + TS types     |
| % % % % api-zod/                   | Generated Zod request validators              |
| % % % % db/                        | Drizzle schema, one file per table            |
| % % % % email/                     | Resend wrapper with dev console-log fallback  |
| % % % % export/                    | PDF generation deps (pdfkit etc.)             |
| % % % % integrations-anthropic-ai/ | Claude wrapper using Replit AI proxy          |
| % % % scripts/                     | Document generators (this PDF included)       |
| % % % docs/                        | Investor-ready PDFs and source markdown       |

## 12.1 Key files for due diligence

| File                                                        | What it is                                      |
|-------------------------------------------------------------|-------------------------------------------------|
| `lib/api-spec/openapi.yaml`                                 | Authoritative API contract                      |
| `artifacts/api-server/src/app.ts`                           | Middleware order, raw-body Stripe mount, routes |
| `artifacts/api-server/src/lib/auth.ts`                      | Clerk !' local-user JIT bridge                  |
| `artifacts/api-server/src/lib/tier.ts`                      | Tier gates, rate limits, escalation bypass      |
| `artifacts/api-server/src/engines/prompts.ts`               | Every system prompt, v2-hardened                |
| `artifacts/api-server/src/engines/f{1..8}*.ts`              | One file per engine                             |
| `artifacts/api-server/src/routes/stripe-webhook.ts`         | Idempotent webhook with rollback semantics      |
| `artifacts/api-server/src/lib/badges.ts`                    | Live + stored badge logic with SSRF guard       |
| `artifacts/api-server/src/lib/ingestion-credits.ts`         | Atomic credit claim with release-on-failure     |
| `lib/db/src/schema/*.ts`                                    | One Drizzle table per file                      |
| `artifacts/command-centre/src/pages/session-detail.tsx`     | Stage selector + workspace switch               |
| `artifacts/command-centre/src/components/<br>workspaces/F*` | One React component per engine                  |

## 13. Roadmap

### Now (shipped, this snapshot)

- F1 !' F8 engines complete, all v2-hardened prompts
- Stripe billing with idempotent webhook and unknown-price guard
- Ingestion engine with per-project Stripe credit
- Quest badges (live + stored) and Context Craft sub-pillars
- Demo Mode for investor previews
- Sentry on both API and web, gated on env

### Next ("d 1 quarter)

- ZIP export endpoint for F8 codebase bundles (currently single-JSON)
- Public exemplar gallery with cert verification links inline
- Self-service team seats on Architect and Institution tiers
- Webhook for third-party integrations to subscribe to cert issuance

### Later

- Private Institution deployments (single-tenant, customer-owned VPC)
- Marketplace for community-authored exemplars (rev share)
- Multi-language Code DJ targets (Python/FastAPI, Go, Rust)



## 14. What an Investor Should Take Away

---

1. **The product is finished, not aspirational.** Every engine in the pipeline ships, end-to-end, with payments, auth, and observability.
2. **The pipeline is the moat.** Eight ordered engines, each one contract-first and certified, is not a feature that a chat-box competitor can backfill in a sprint.
3. **The business model is mixed and defensible.** Recurring subscription plus a low-friction one-time ingestion credit plus an enterprise tier covers every buyer profile we have encountered.
4. **The engineering posture is mature.** Idempotent webhooks, external- first deletions, SSRF guards, strict JIT sync, telemetry as best-effort — these are the marks of a team that has shipped before, not first-time founders learning on the job.
5. **The cap-table cost of operations is low.** No customer API keys to handle. No infrastructure to procure. One-click deploy on Replit. Operational surface area is deliberately small so engineering can keep shipping product.

---

*Generated from the codebase-of-record on the date of issue. The HARNESS session that produced this document was a manual F1 !' F7 run, then hand-edited for investor narrative. The underlying technical content is auto-synchronised with `replit.md` — the file that travels with the repository.*